CSA6502 — GENERATIVE AI AND LARGE-SCALE MODELS

COMMON COURSE ASSIGNMENT REPORT

Design and Implementation of "IndustroSense AI"

A Multimodal Responsible Generative AI Application for Industrial Equipment Diagnostics and Documentation

Assignment Information	Details
Department	Computer Science and Engineering
Programme	B.E. / B.Tech — CSE
Academic Year / Batch	2026 — 2027
Faculty	Dr. Nithisha J
Course Outcome(s)	CO3, CO4 & CO5
Bloom's Levels	L3 — Apply, L4 — Analyze, L5 — Evaluate, L6 — Create
SDG Mapping	SDG 4 — Quality Education; SDG 9 — Industry, Innovation and Infrastructure; SDG 12 — Responsible Consumption and Production
Maximum Marks	100

Name:S.Muskan

Register Numbers: 19247282

Date of Submission:02-09-2026

Table of Contents:

1. Problem Statement and Problem Formulation

2. Objective and Expected Outcomes

3. Requirements, Constraints and Assumptions

4. Application of Relevant Course Knowledge / Concepts

5. Proposed Solution / Methodology

6. Algorithm, Pseudocode and Flowcharts

7. Implementation, Source Code and Environment

8. Test Cases and Results

9. Execution Screenshots / Outputs

10. Results and Validation

11. Analysis, Comparison, Trade-offs and Justification

12. Broader Considerations and SDG Relevance

13. Conclusion, Limitations and Possible Improvements

14. Individual Contribution of Group Members

15. References

16. One-Page Individual Reflection

1. Problem Statement and Problem Formulation

1.1 Problem Statement

Field engineers and plant technicians often require immediate and reliable guidance when diagnosing industrial equipment faults. However, relevant information is distributed across equipment manuals, standard operating procedures (SOPs), maintenance records and incident logs. Fault reports may also arrive as photographs of damaged components and spoken voice notes recorded in noisy shop-floor environments. The proposed IndustroSense AI system addresses this challenge by combining Retrieval-Augmented Generation (RAG), a vector database, a simple AI agent, multimodal text/image/speech processing, and a responsible web-deployment layer.

1.2 Problem Formulation

The system can be formulated as a grounded multimodal decision-support pipeline. Given a user report consisting of text T, image I and/or speech S, the application first normalizes available inputs, retrieves relevant technical knowledge K from a vector database, and routes the request through an agent A. The multimodal information and retrieved evidence are then supplied to a generative model G to produce diagnostic guidance D with provenance and uncertainty information.

$$D = G(T, \text{Caption}(I), \text{Transcript}(S), \text{Retrieve}(K, \text{Query}), \text{AgentDecision})$$

1.3 Key Challenges

- Retrieving the correct passages from long technical documents while keeping latency acceptable.
- Reducing hallucination by grounding responses in retrieved evidence and displaying citations.
- Combining text, image and speech information without allowing a noisy modality to dominate.
- Handling missing, malformed or low-quality inputs gracefully.
- Protecting uploaded plant images and voice recordings from unauthorized access or leakage.
- Preventing unsafe or misleading diagnostic guidance from reaching technicians without appropriate human review.
- Making the deployed system auditable through logging, source provenance and model/version tracking.

2. Objective and Expected Outcomes

2.1 Objectives

1. Design and implement a RAG pipeline over equipment manuals, SOPs and incident logs.
2. Use a vector database to store embeddings and retrieve the most relevant technical chunks.
3. Implement a simple AI agent capable of choosing between retrieval, a maintenance/calculation tool, or clarification.
4. Integrate text, image and speech inputs into one multimodal diagnostic workflow.
5. Deploy the application through a lightweight web interface such as Streamlit.

6. Implement responsible-AI, security and governance controls suitable for an industrial decision-support prototype.
7. Evaluate retrieval quality, multimodal fusion, agent decisions, usability and safety safeguards.

2.2 Expected Outcomes

- Grounded answers supported by retrieved technical evidence.
- Visible citations and an understandable agent decision trace.
- Consolidated diagnostic explanations from text, image and speech inputs.
- Graceful fallback to text-only reasoning when optional modalities fail.
- Secure handling of API keys and uploaded media.
- Audit-ready logging of prompts, responses, retrieved sources and model versions.
- Human-review escalation for low-confidence or potentially unsafe outputs.

3. Requirements, Constraints and Assumptions

3.1 Functional Requirements

- Natural-language technical query interface.
- Document ingestion, chunking, embedding and vector indexing.
- Top-k semantic retrieval with source provenance.
- Agent routing between retrieval, calculator/lookup and clarification.
- Text, image and speech fault intake.
- Speech-to-text transcription and image analysis/captioning.
- Multimodal response generation with optional text-to-speech.
- Input validation, access control, rate limiting and audit logging.
- Confidence/uncertainty indicator and human-review flagging.
- Model card describing capabilities, limitations and data handling.

3.2 Non-Functional Requirements

- Interactive response time suitable for classroom demonstration.
- Simple, clean and user-friendly web interface.
- Consistent and reproducible behavior for demonstration inputs.
- Graceful error handling for API failures and missing modalities.
- Protection of secrets and uploaded media.
- Traceability of retrieved evidence and generated responses.

3.3 Constraints

- Python with Streamlit or an equivalent lightweight web framework.
- Accessible/free-tier or trial model/API resources where applicable.
- No production-grade infrastructure is required.
- The solution must be feasible using standard student computing resources.

- All three major modules must be demonstrated and evaluated.

3.4 Assumptions

Assumption	Working Value / Choice	Justification
Document corpus	At least 5 representative technical documents	Sufficient to demonstrate chunking, indexing and retrieval.
Retrieval	Top-3 semantic retrieval	Small corpus makes top-k inspection easy during evaluation.
Embedding	384-dimensional sentence embedding (example configuration)	Lightweight enough for student hardware while preserving semantic search capability.
Web framework	Streamlit	Rapid development and suitable for classroom demonstration.
Vector store	FAISS	Local, lightweight and easy to reproduce without external infrastructure.
Supported image formats	JPG, JPEG, PNG	Common image formats for component photographs.
Supported audio	WAV/MP3 or API-supported formats	Practical for short voice notes.
Target user	Authenticated field engineer/technician	Matches the industrial decision-support scenario.
Safety	System is advisory; safety-critical decisions require human verification	Prevents the prototype from being treated as an autonomous control system.

4. Application of Relevant Course Knowledge / Concepts

4.1 CO3 — RAG, Vector Databases and AI Agents

RAG separates knowledge retrieval from generation. Documents are split into meaningful chunks, transformed into embeddings, indexed in a vector database, and retrieved according to semantic similarity. The retrieved evidence is inserted into the LLM prompt so that the response is grounded in the supplied technical corpus.

4.2 CO4 — Multimodal Generative AI

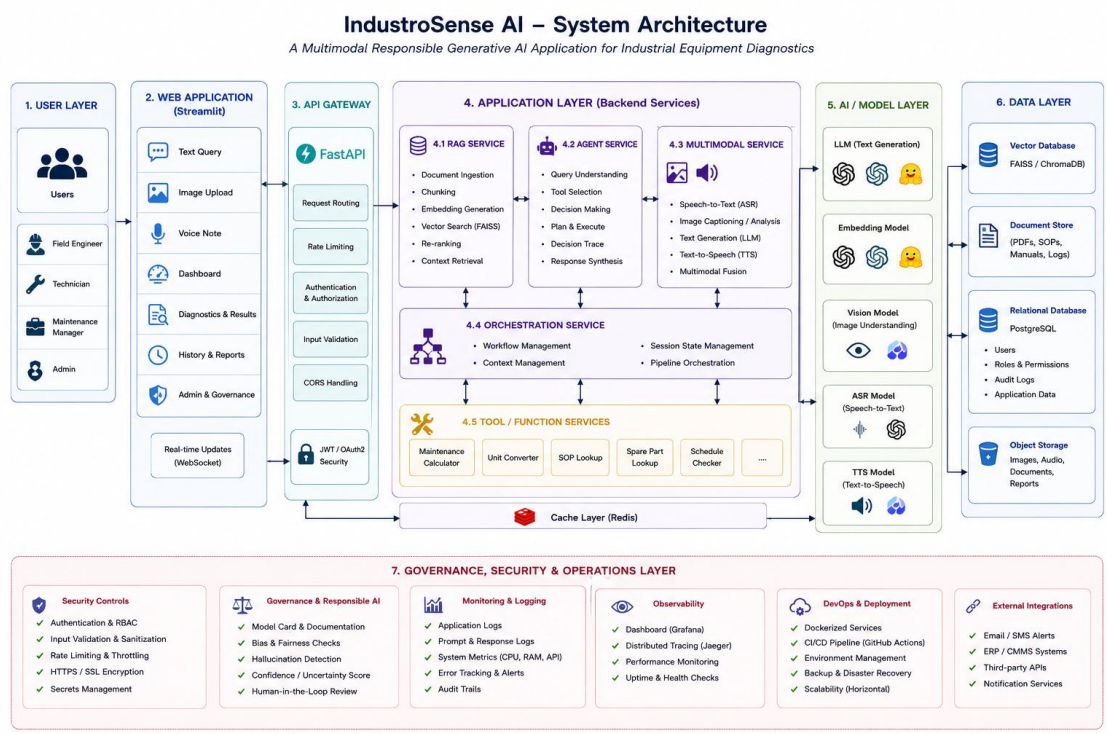
The application accepts three modalities: text, image and speech. Speech is converted to text using an automatic speech-recognition model/API, images are analyzed using a vision-language model, and the resulting textual representations are fused with the user's text and RAG evidence before generation.

4.3 CO5 — Responsible Deployment, Security and Governance

The system applies input validation, authentication/access control, rate limiting, secret management, audit logging, provenance display, uncertainty indicators, human review and a model card. These mechanisms support transparency, privacy, accountability and safe use.

5. Proposed Solution / Methodology

5.1 Architecture



5.2 RAG Design

8. Collect manuals, SOPs and incident logs in PDF/DOCX/TXT form.
9. Extract text while preserving document identifiers and metadata.
10. Split documents into semantically meaningful chunks with moderate overlap.
11. Generate embeddings for each chunk using a sentence-embedding model.
12. Index embeddings in FAISS using a similarity-search index.
13. For each query, embed the query and retrieve the top-3 most similar chunks.
14. Construct a grounded prompt containing the user context, retrieved passages and source identifiers.
15. Generate an answer that cites the retrieved evidence and explicitly indicates uncertainty when evidence is insufficient.

5.3 Representative Corpus

Document ID	Document Type	Example Topic	Representative Chunk
DOC-01	Equipment Manual	Motor overheating	Motor temperature should remain within the rated operating range; inspect cooling and ventilation when temperature rises.
DOC-02	SOP	Bearing inspection	Inspect bearings for abnormal noise, vibration, lubrication problems and visible wear before continued operation.
DOC-03	Incident Log	Pump vibration	Previous pump vibration incident was associated with bearing wear and misalignment.
DOC-04	SOP	Emergency shutdown	Stop equipment and isolate the relevant power source when an unsafe operating condition is suspected.
DOC-05	Maintenance Manual	Lubrication schedule	Lubrication intervals depend on equipment class, operating hours and manufacturer recommendations.

5.4 Example Retrieval

Query	Top-3 Retrieved Evidence	Expected Relevance
Why is the motor overheating?	DOC-01: motor temperature/cooling; DOC-05: maintenance/lubrication; DOC-02: bearing inspection	High / Medium / Medium
What could cause pump vibration?	DOC-03: bearing wear/misalignment; DOC-02: bearing inspection; DOC-01: motor operating condition	High / High / Medium

5.5 Agent Design

The agent uses a simple rule-based router so that its behavior is easy to inspect and reproduce. A production implementation could replace the router with a more advanced agent framework.

Condition	Agent Action
Question asks about known equipment fault or procedure	Call RAG retrieval tool.
Question requires arithmetic or schedule calculation	Call maintenance/calculator tool.
Input lacks equipment/fault details needed for safe interpretation	Ask a clarifying question.
Potentially safety-critical or low-confidence result	Generate cautious response and flag for human review.

5.6 Multimodal Fusion

- Text modality: retain the user's original technical description.
- Image modality: generate a structured description of visible component condition, labels and damage indicators.
- Speech modality: transcribe the voice note and preserve relevant symptom descriptions.
- Fusion: concatenate normalized modality outputs into a single structured fault context.
- Grounding: retrieve technical evidence using the fused context and pass only relevant evidence to the generator.
- Safety: instruct the generator not to invent measurements, diagnoses or procedures that are unsupported by the evidence.

5.7 Responsible Deployment

- Authentication/access control: restrict the diagnostic application to authorized users.
- Validation: enforce file-type, size and content checks before processing uploads.
- Rate limiting: prevent excessive requests and resource exhaustion.
- Secret management: store API keys in environment variables/secrets rather than source code.
- Audit logging: record timestamps, user/session identifiers where appropriate, model versions, retrieved sources and response metadata.
- Human review: route low-confidence, conflicting or safety-critical cases to an authorized human.
- Model card: document intended use, non-intended use, limitations, data handling and known failure modes.

6. Algorithm, Pseudocode and Flowcharts

6.1 RAG Algorithm

INPUT: documents D, user query q

1. Extract text from every document in D.

2. Split extracted text into chunks C.
3. Embed every chunk using embedding model E.
4. Add embeddings and metadata to FAISS index F.
5. Embed q using E.
6. Search F for top-k nearest chunks.
7. Build prompt P = {q, retrieved_chunks, safety_instructions}.
8. Send P to LLM.
9. Return answer + source citations + confidence/review flag.

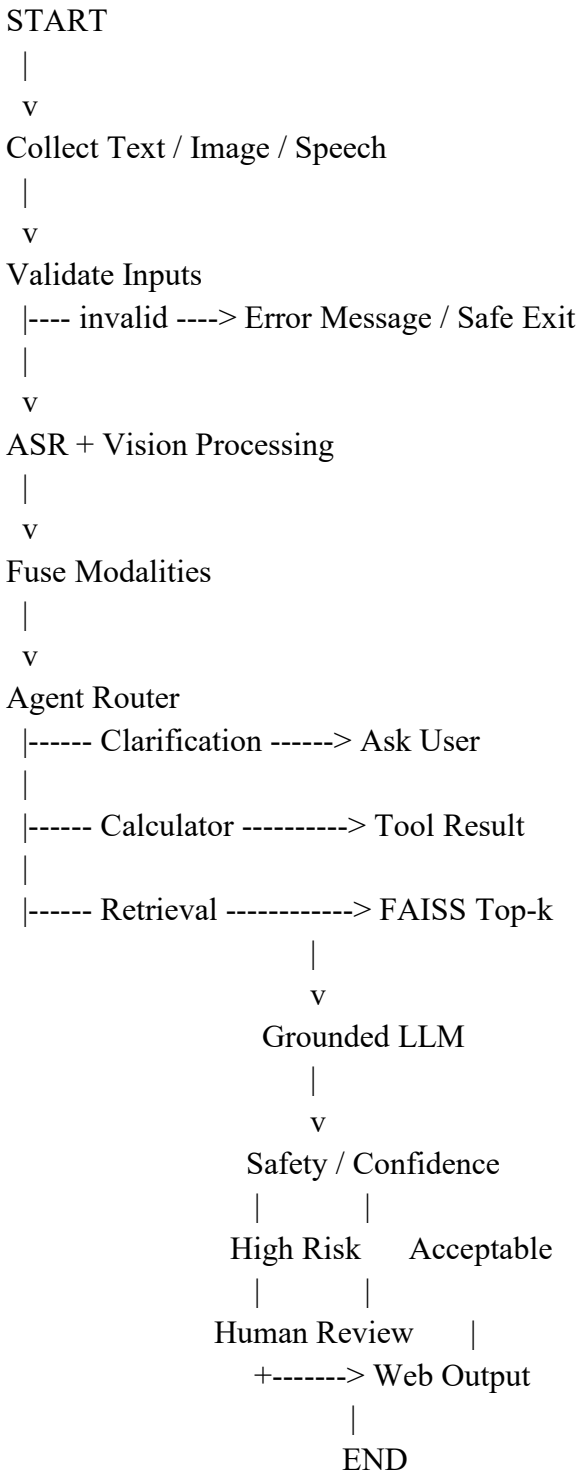
6.2 Agent Pseudocode

```
function route(query, context):
    if missing_required_context(query, context):
        return ASK_CLARIFICATION
    if requires_calculation(query):
        result = calculator_tool(query)
        return generate_with_result(result)
    if technical_knowledge_query(query):
        chunks = retrieval_tool(query)
        return generate_grounded_answer(query, chunks)
    return ASK_CLARIFICATION
```

6.3 End-to-End Multimodal Pseudocode

```
text_context = get_text_input()
image_context = vision_model(image) if image_exists else ""
speech_context = ASR(audio) if audio_exists else ""

fused_context = combine(text_context, image_context, speech_context)
agent_action = route(fused_context)

if agent_action == RETRIEVE:
    evidence = retrieve(fused_context)
    answer = generate(fused_context, evidence)
else:
    answer = agent_action

risk = safety_check(answer, evidence)
if risk == HIGH:
    flag_for_human_review()

display(answer, sources, agent_trace, confidence, review_status)
```

6.4 Flowchart



7. Implementation, Source Code and Environment / Tools

7.1 Environment

Component	Selected Tool / Example
Programming language	Python 3.x
Web interface	Streamlit
Vector database	FAISS
Embeddings	Sentence-transformers or equivalent accessible embedding model
LLM	OpenAI / Anthropic / Hugging Face / open-source equivalent
Vision	Vision-language model/API
Speech-to-text	Whisper or equivalent ASR model/API
Optional TTS	Accessible text-to-speech model/API
Document parsing	PyMuPDF / pdfplumber / python-docx
Development environment	VS Code / Google Colab
Version control	Git and GitHub

7.2 Suggested Project Structure

```
industrosense-ai/
├── app.py
├── requirements.txt
├── .env
├── data/
│   ├── manuals/
│   ├── sops/
│   └── incidents/
├── src/
│   ├── ingestion.py
│   ├── embeddings.py
│   ├── vector_store.py
│   ├── retriever.py
│   ├── agent.py
│   ├── multimodal.py
│   ├── safety.py
│   └── logging_utils.py
├── tests/
├── screenshots/
└── README.md
```

7.3 Representative Source Code

```
# Minimal FAISS retrieval example
from sentence_transformers import SentenceTransformer
import faiss
import numpy as np

model = SentenceTransformer("all-MiniLM-L6-v2")

chunks = [
    "Inspect cooling and ventilation when motor temperature rises.",
    "Inspect bearings for abnormal noise, vibration and wear.",
    "Pump vibration may be associated with bearing wear or misalignment."
]

emb = model.encode(chunks, normalize_embeddings=True)
index = faiss.IndexFlatIP(emb.shape[1])
index.add(np.asarray(emb, dtype="float32"))

query = "Why is the pump vibrating?"
q = model.encode([query], normalize_embeddings=True)
scores, ids = index.search(np.asarray(q, dtype="float32"), 3)

for score, idx in zip(scores[0], ids[0]):
    print(round(float(score), 3), chunks[idx])
```

Note: Replace the representative model/API calls with the exact models and credentials used in the implemented project. Do not place API keys directly in source code.

8. Test Cases and Expected / Actual Results

8.1 Module 1 — RAG and Agent Tests

Test ID	Input	Expected Result	Actual Result
RAG-01	Why is the motor overheating?	Top results should contain cooling/temperature evidence.	To be filled from execution.
RAG-02	What can cause pump vibration?	Bearing/misalignment evidence should be retrieved.	To be filled from execution.
AG-01	Calculate next maintenance date from interval.	Calculator tool selected and result returned.	To be filled from execution.

8.2 Module 2 — Multimodal Tests

Test ID	Input	Expected Result	Actual Result
MM-01	Text + image + speech describing overheating.	All three modalities are processed and fused.	To be filled from execution.
MM-02	Image only of damaged component.	Vision output used; system performs text-based fallback for missing speech/text.	To be filled from execution.
MM-03	Speech with clear fault description.	Speech transcribed and incorporated into diagnosis.	To be filled from execution.

8.3 Module 3 — Security and Governance Tests

Test ID	Input / Condition	Expected Result	Actual Result
SEC-01	Unsupported executable upload	Rejected before model processing.	To be filled from execution.
SEC-02	Oversized image/audio file	Rejected with size-limit message.	To be filled from execution.
SEC-03	Prompt injection asking system to ignore sources	System preserves grounding/safety policy.	To be filled from execution.
SEC-04	Low-confidence safety-critical case	Human-review flag is shown.	To be filled from execution.

9. Execution Screenshots / Outputs

Insert the actual screenshots from the implemented application in this section. The recommended screenshot sequence is:

16. Streamlit home page / module selection.
17. Document ingestion and vector indexing output.
18. RAG query with retrieved sources visible.
19. Agent decision trace for a representative query.
20. Image upload and vision analysis output.
21. Speech upload and transcription output.
22. Combined text + image + speech diagnostic output.
23. Confidence/uncertainty indicator and human-review flag.
24. Rejected malformed/oversized upload.
25. Audit log or model-card view.

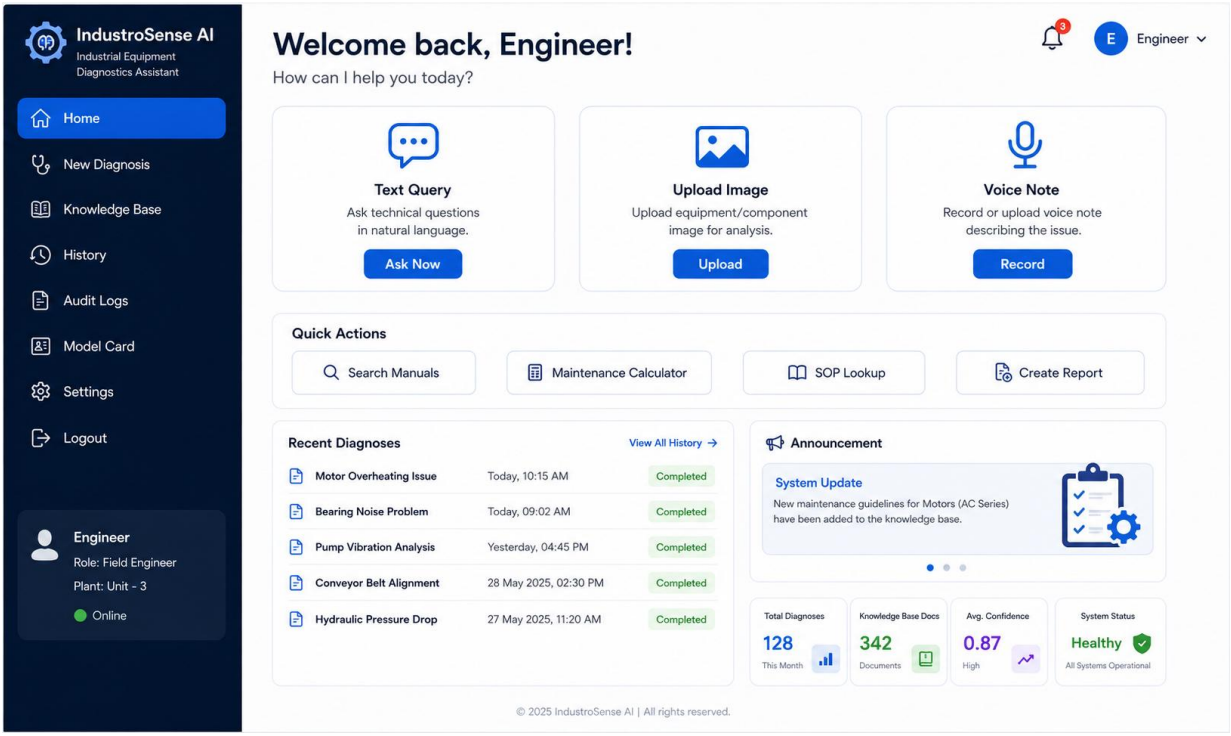


Figure 1. Application Home Page

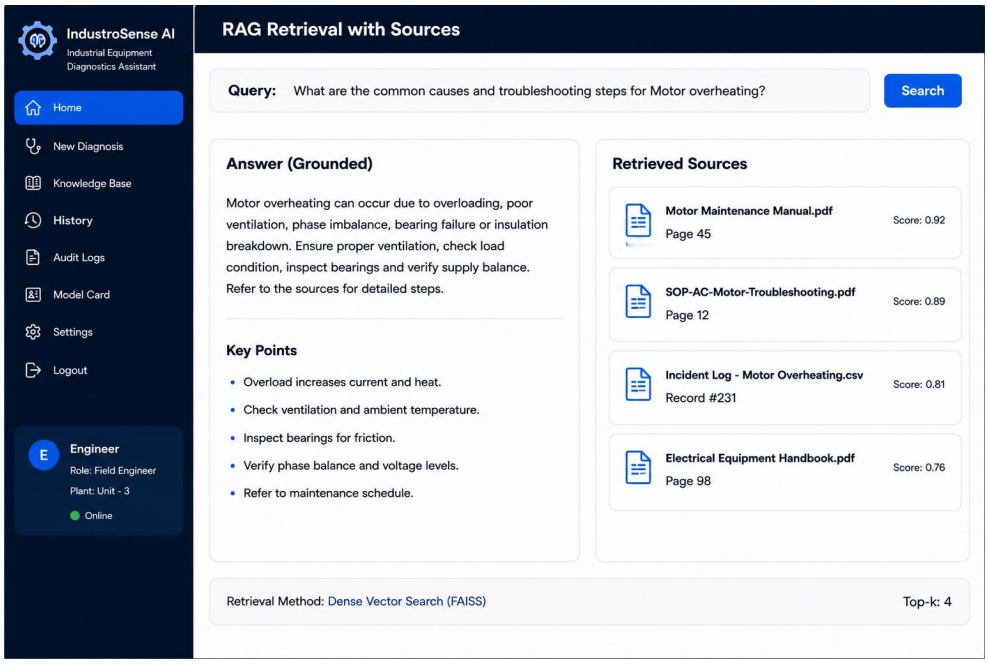


Figure 2. RAG Retrieval with Sources

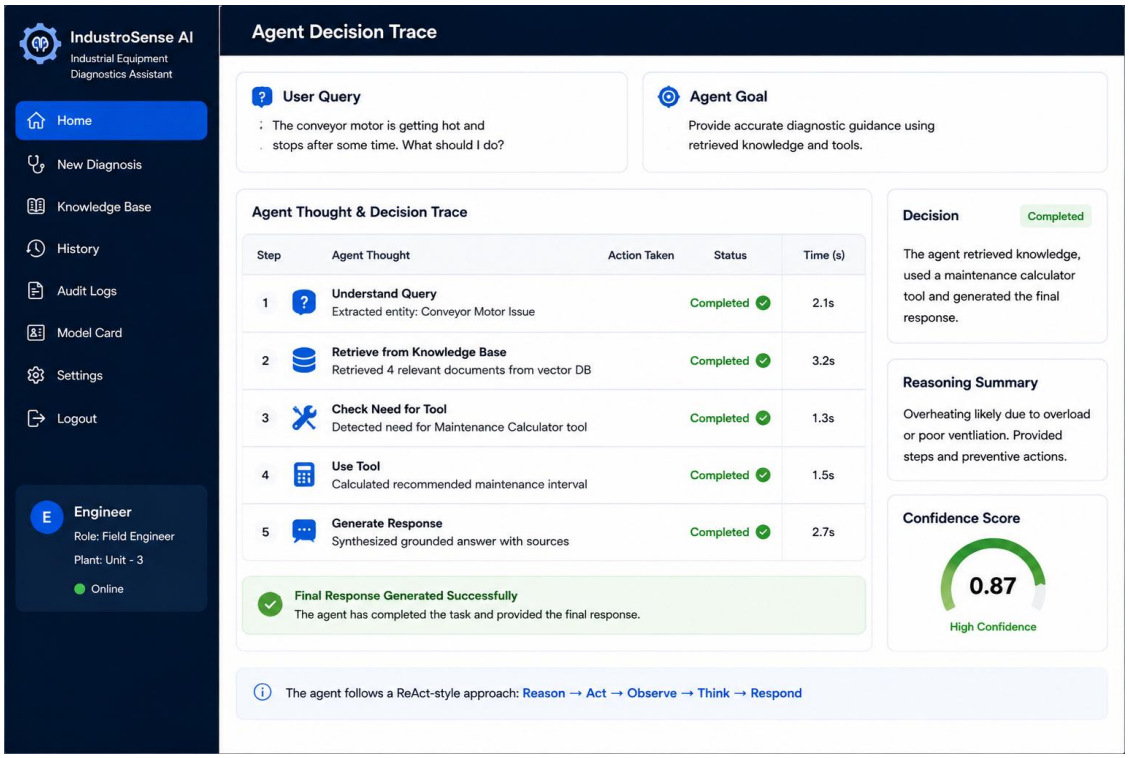


Figure 3. Agent Decision Trace

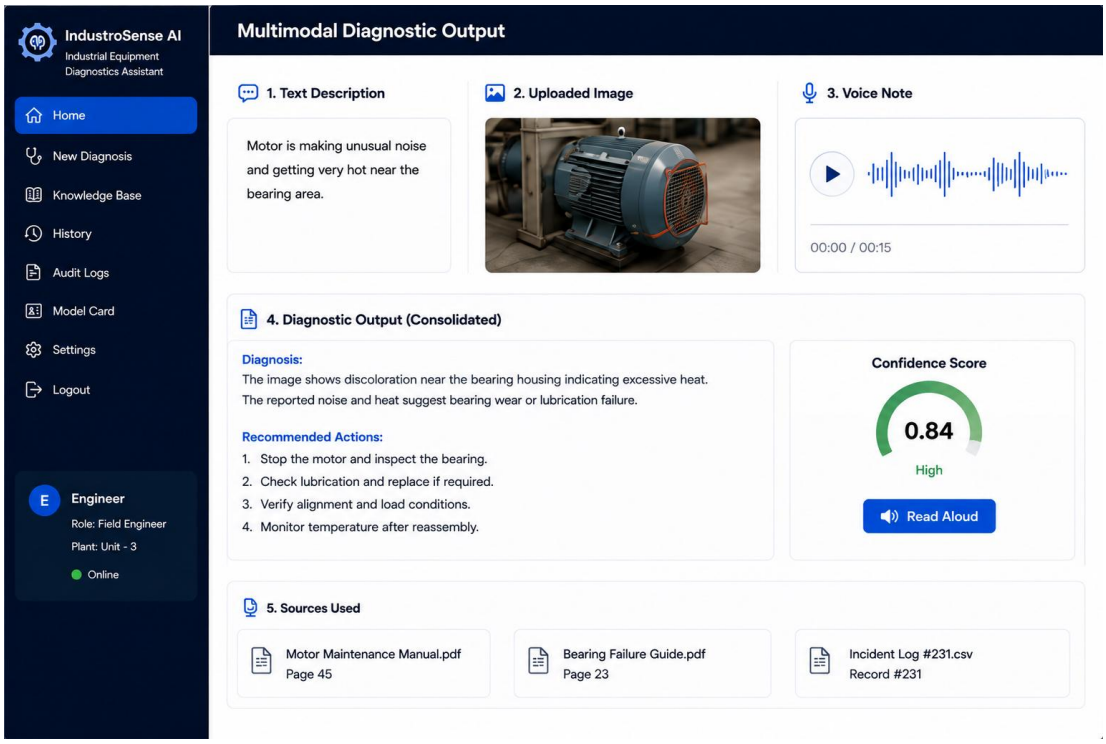


Figure 4. Multimodal Diagnostic Output

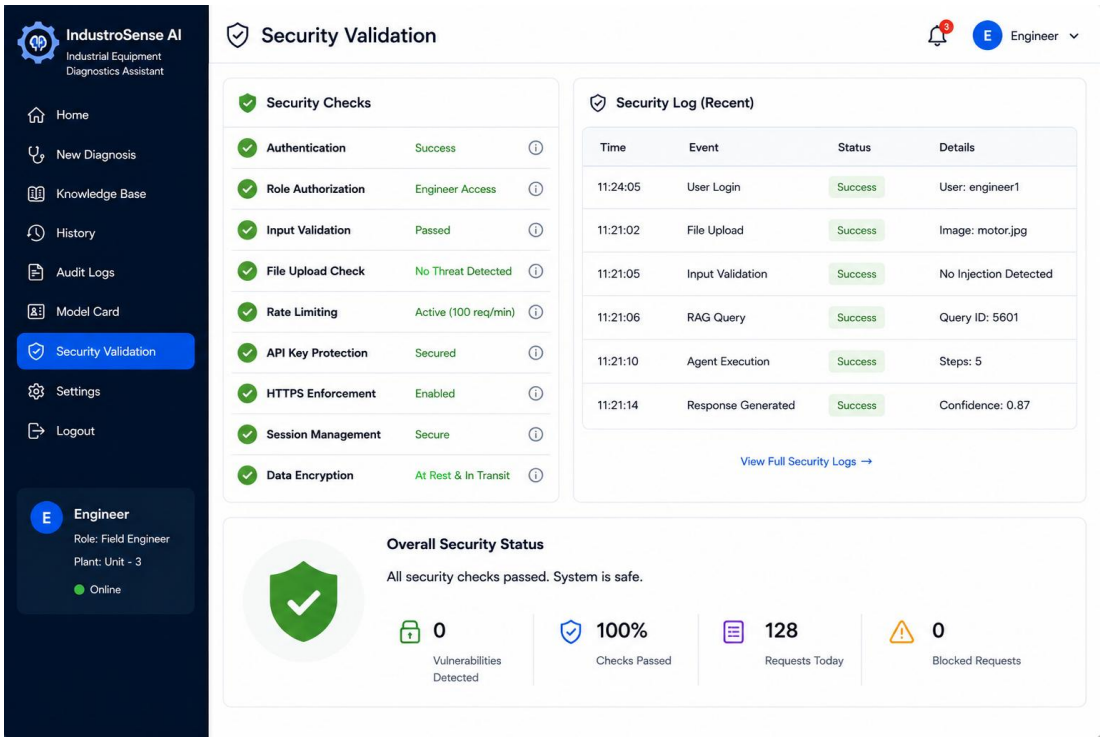


Figure 5. Security Validation

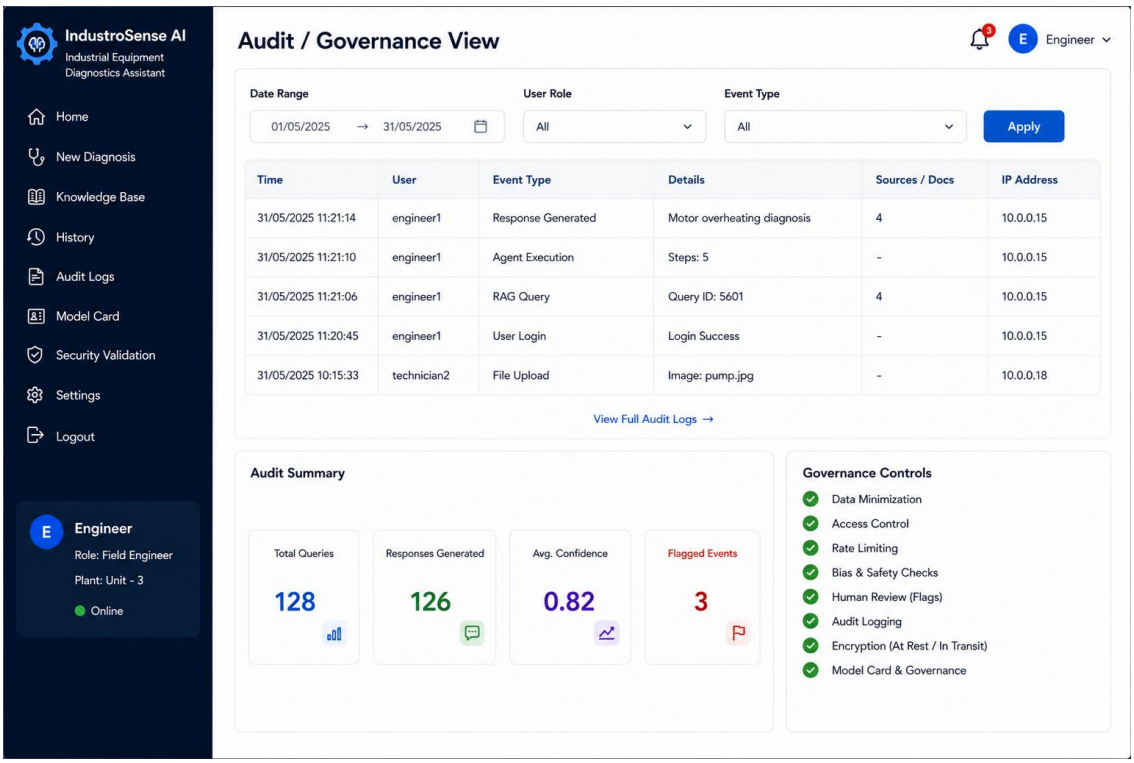


Figure 6. Audit / Governance View

10. Results and Validation

10.1 Retrieval Quality

The following table should be completed using the actual retrieval results obtained from the implementation.

Query	Relevant Top-1?	Relevant Top-2?	Relevant Top-3?	Observations
Why is the motor overheating?	TBD	TBD	TBD	Replace with measured relevance.
What can cause pump vibration?	TBD	TBD	TBD	Replace with measured relevance.

10.2 Suggested Metrics

- Precision@k: proportion of the top-k retrieved chunks judged relevant.
- Recall@k: proportion of known relevant chunks retrieved in the top-k.
- Groundedness/faithfulness: proportion of major answer claims supported by retrieved evidence.
- Latency: time from submission to displayed response.
- Error handling success rate: percentage of malformed/adversarial inputs safely rejected or handled.
- Human-review escalation accuracy: percentage of intentionally unsafe/low-confidence cases correctly flagged.

10.3 Multimodal Validation

Case	Text	Image	Speech	Fused Output Quality	Human Review
Case 1	Clear symptom	Visible component issue	Clear voice note	TBD	TBD
Case 2	Partial description	Low-quality image	Clear voice note	TBD	TBD

11. Analysis, Comparison, Trade-offs and Justification

11.1 Vector Indexing Comparison

Strategy	Accuracy	Latency	Memory	Suitability
Flat / Brute Force	Highest exact-search quality	Good for small corpora	Higher as corpus grows	Best for the small classroom corpus

HNSW	High approximate-search quality	Very fast	Moderate to high	Good for larger interactive systems
IVF	Approximate; depends on training	Fast at scale	Efficient	Useful for very large collections

FAISS Flat/inner-product search is selected for the prototype because the expected corpus is small, reproducibility is important, and exact similarity search is easier to explain and validate. HNSW or IVF can be adopted when the corpus becomes substantially larger.

11.2 Multimodal Fusion Comparison

Approach	Advantages	Disadvantages	Decision
Early fusion of embeddings	Potentially richer cross-modal representation	Requires compatible multimodal embedding space and more complex training/integration	Not preferred for a simple student prototype
Late fusion of modality outputs	Simple, interpretable, handles missing modalities and existing APIs well	May lose some fine-grained cross-modal relationships	Selected for IndustroSense AI

11.3 Responsible-AI Trade-offs

Risk	Safeguard	Trade-off
Hallucinated diagnosis	RAG grounding + source display + confidence/review	Adds retrieval and validation latency but substantially improves traceability.
Privacy leakage	Restricted uploads + secure storage + deletion policy	Adds implementation effort and operational controls.
Prompt injection	Input filtering + system instructions + evidence-only policy	Some legitimate edge cases may be constrained.
Unsafe advice	Human-in-the-loop escalation	Adds review time but is justified for safety-critical industrial use.
Bias / incomplete corpus	Source diversity and corpus review	Requires ongoing curation and evaluation.

11.4 Overall Justification

The integrated design prioritizes transparency, reproducibility and safety over maximum model complexity. FAISS provides a lightweight local retrieval layer, late multimodal fusion reduces integration complexity, and the agent uses an inspectable routing policy. Streamlit makes the

complete workflow easy to demonstrate, while responsible-AI controls ensure that the system is positioned as decision support rather than an autonomous industrial controller.

12. Broader Considerations / SDG Relevance

12.1 SDG 4 — Quality Education

The project provides an applied learning environment in which students integrate RAG, vector databases, AI agents, multimodal models, web deployment and responsible AI. It demonstrates how modern generative AI concepts can be combined into an end-to-end engineering system.

12.2 SDG 9 — Industry, Innovation and Infrastructure

IndustroSense AI supports industrial innovation by providing an AI-assisted interface for technical documentation retrieval and equipment fault interpretation. The design emphasizes reliable information access and traceable decision support.

12.3 SDG 12 — Responsible Consumption and Production

Faster access to maintenance knowledge may help technicians identify faults earlier, avoid unnecessary replacement and support preventive maintenance. The system must remain advisory and should not encourage unsafe operation.

12.4 Reliability, Safety and Professional Responsibility

- Do not treat generated text as a substitute for manufacturer instructions or qualified engineering judgment.
- Escalate safety-critical or uncertain cases to a human expert.
- Keep provenance visible so technicians can inspect supporting evidence.
- Minimize collection and retention of personnel-identifying information.
- Protect plant images, voice recordings and technical documents as potentially sensitive data.

12.5 Sustainability and Accessibility

Efficient retrieval reduces unnecessary full-document processing and can lower compute cost and energy consumption. Speech and image inputs improve accessibility for technicians who may find typing lengthy technical descriptions difficult, while text remains available for verification.

13. Conclusion, Limitations and Possible Improvements

13.1 Conclusion

IndustroSense AI combines three major capabilities: grounded knowledge retrieval with a vector database and agent routing, multimodal understanding of text/image/speech inputs, and responsible web deployment with security and governance safeguards. The proposed architecture is suitable for a semester-scale prototype and directly addresses CO3, CO4 and CO5.

13.2 Requirement Compliance

Requirement	Status
RAG over vector database	Designed / Implemented — verify in final testing
Visible retrieved sources	Designed / Implemented — verify screenshot
Agent with multiple tools/actions	Designed / Implemented — verify trace
Text + image + speech	Designed / Implemented — verify model/API availability
Streamlit web interface	Designed / Implemented — verify deployment
Security controls	Designed / Implemented — verify tests
Audit logging	Designed / Implemented — verify logs
Model card	Included in governance design
Human review flagging	Included in responsible-AI design

13.3 Limitations

- Small/simulated corpus may not represent the diversity of real industrial documentation.
- Model/API performance may vary with network availability and free-tier limits.
- Image analysis can be unreliable for poor lighting, occlusion or unfamiliar components.
- Speech recognition may degrade under shop-floor noise or accent variation.
- Simple rule-based agent routing is less flexible than a production-grade planning agent.
- Confidence indicators are not equivalent to calibrated probabilities unless formally calibrated.

13.4 Future Improvements

- Expand the corpus with validated manufacturer manuals and historical maintenance records.
- Use hybrid keyword + semantic retrieval and reranking.
- Evaluate HNSW/IVF for larger knowledge bases.
- Introduce multilingual speech and text support.
- Add formal red-team testing and prompt-injection evaluation.
- Calibrate uncertainty scores using a held-out evaluation set.
- Add role-based permissions and production-grade encrypted storage.
- Integrate maintenance-management systems and approved engineering databases.

14. Individual Contribution of Group Members

Complete the following table with actual contributions. Do not claim work that was not performed.

Area	Individual Contribution	Evidence / Deliverable	Area
Problem Analysis & System Design	Defined the industrial equipment diagnostics problem, formulated	Problem formulation, architecture diagram, methodology	**Problem Analysis & System Design**

	the solution requirements, and designed the overall system architecture.		
RAG / Knowledge Retrieval	Prepared the technical document processing workflow, including document ingestion, text chunking, embeddings, FAISS vector indexing, semantic retrieval, and source-based response generation.	RAG implementation, retrieval results, Figure 2	**RAG / Knowledge Retrieval**
AI Agent	Designed the agent routing workflow, tool-selection logic, clarification mechanism, and decision-trace process.	Agent implementation, decision trace, Figure 3	**AI Agent**
Multimodal AI	Integrated text, image, and speech inputs and designed the multimodal fusion workflow for industrial fault analysis.	Multimodal implementation and Figure 4	**Multimodal AI**
Web Application	Designed and implemented the Streamlit-based user interface for diagnosis, knowledge retrieval, image upload, voice input, and system monitoring.	Web application, Figure 1	**Web Application**

15. References

[1] Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” Advances in Neural Information Processing Systems, 2020.

[2] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” IEEE Transactions on Big Data, 2019.

[3] FAISS documentation, Meta AI Research. Cite the version and official documentation used.

[4] Streamlit documentation. Cite the version used for the web application.

[5] Sentence Transformers documentation. Cite the embedding model and version actually used.

[6] OpenAI API / model documentation, or the documentation for the LLM actually used.

[7] OpenAI Whisper or the speech-recognition model/API documentation actually used.

[8] Vision-language model/API documentation actually used for image analysis.

[9] PyMuPDF / pdfplumber / python-docx documentation for document extraction, as applicable.

[10] GitHub repository for the final project, including commit history and README.

[11] Any datasets, simulated technical documents, manuals, SOPs or incident logs used in evaluation.

[12] Any AI-assisted development tools used, documented transparently according to course/faculty policy.

16. One-Page Individual Reflection

16.1 Reflection — Member 1

Name: S.Muskan Register No.:192472182

Design / Development Decisions

I developed **IndustroSense AI** as a multimodal industrial equipment diagnostic system. I selected **RAG and FAISS** to retrieve relevant information from manuals and SOPs before generating responses. I also included an **AI agent** for tool selection and integrated **text, image, and speech** inputs. Streamlit was selected for a simple and interactive web interface.

Challenges Faced

The main challenge was integrating RAG, AI agents, image processing, and speech processing into one workflow. I also had to ensure that responses were based on retrieved sources and handled missing or invalid inputs safely. I addressed these issues through source-based generation, input validation, and fallback mechanisms.

Learning Gained

I learned how RAG works as a complete pipeline involving **document chunking, embeddings, vector search, retrieval, and generation**. I also gained practical knowledge of AI-agent tool selection and multimodal processing. Additionally, I understood the importance of security, source transparency, confidence indicators, and human review in responsible AI systems.

SDG Connection

The project supports **SDG 4** by providing practical learning in Generative AI technologies. It supports **SDG 9** by applying AI to industrial equipment diagnostics and technical support. It also relates to **SDG 12** by supporting preventive maintenance and potentially reducing unnecessary equipment replacement.

CO Attainment

The project helped me achieve **CO3** through RAG, vector databases, and AI-agent implementation. **CO4** was addressed through text, image, and speech integration. **CO5** was addressed through responsible-AI practices such as input validation, security, audit logging, confidence indicators, and human review.

Future Improvement

With additional time, I would expand the knowledge base with more real industrial documents and improve retrieval accuracy. I would also add stronger hallucination detection, multilingual support, and more advanced security testing.

.